Executions · Abdallah

GIU Curriculum and Academic Innovation Management Platform (GCAIMP) · Milestone 2 · executions divided between the team, same split as Procedures.pdf

From the Milestone 2 document (GIU, Winter 2026, submission 15/11/2026, 11:59 pm):

d) **Executions.sql**: an example execution of every stored procedure and function (**EXEC** for procedures, **SELECT** for functions), using data from your insertions.

• The four SQL files must run in this order without errors: Tables.sql, Insertions.sql, Procedures.sql, Executions.sql.

• It is the students' responsibility to make sure that every stored procedure and function executes correctly and returns the expected output.

• Your procedures and functions must handle any errors or exceptions that may happen (e.g., invalid input, records that do not exist, duplicate records, or violated constraints) and show a clear message instead of failing.

How we do it: write your calls in `executions/<YourName>.sql` on your branch, one execution per procedure or function below, in the same order. Use data from Insertions.sql. Add a second call that triggers an error case where it makes sense. Merge into main and the CI glues it into Executions.sql.

Abdallah 23 executions (one per procedure / function)

2 As an Academic Instructor, I should be able to:

13. Register a new innovation initiative for a course offering, optionally linked to the assessment that motivated it.
    Signature:
    Name: RegisterInnovationInitiative.
    Type: Stored procedure.
    Input: @SubmitterID int, @FacultyID int, @Title varchar(150), @Description varchar(1000), @Motivation varchar(1000), @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20), @TargetStartSemesterID int, @ImplementationDeadline date, @AssessmentID int.
    Output: Confirmation message or new @InitiativeID int.

18. Assign academic members or teaching assistants to an initiative with a role (e.g., Lead, Contributor, Supporting TA).
    Signature:
    Name: AssignInitiativeMember.
    Type: Stored procedure.
    Input: @InitiativeID int, @MemberID int, @Role varchar(50).
    Output: Confirmation message.

20. Add an implementation milestone to an initiative (the sequence number is assigned automatically).
    Signature:
    Name: AddMilestone.
    Type: Stored procedure.
    Input: @InitiativeID int, @Title varchar(150), @Description varchar(1000), @DueDate date.
    Output: Confirmation message showing the new @SeqNo int.

21. Mark a milestone as completed.

Signature:
Name: CompleteMilestone.
Type: Stored procedure.
Input: @InitiativeID int, @SeqNo int, @CompletionDate date.
Output: Confirmation message.

25. Request approval of an initiative from my faculty's dean.
    Signature:
    Name: RequestInitiativeApproval.
    Type: Stored procedure.
    Input: @InitiativeID int, @MemberID int.
    Output: Confirmation message.

26. Track the progress of an initiative's milestones.
    Signature:
    Name: ViewInitiativeProgress.
    Type: Function.
    Input: @InitiativeID int.
    Output: Table showing each milestone, its due date, completion date, status and whether it is overdue.

27. Calculate the completion percentage of an initiative.
    Signature:
    Name: CalculateInitiativeCompletion.
    Type: Function.
    Input: @InitiativeID int.
    Output: Percentage of completed milestones decimal(5,2).

3 As a Teaching Assistant, I should be able to:

6. Update the status of a milestone in an initiative I am assigned to.
   Signature:
   Name: UpdateMilestoneStatus.
   Type: Stored procedure.
   Input: @InitiativeID int, @SeqNo int, @Status varchar(50).
   Output: Confirmation message.

5 As a Dean, I should be able to:

1. View all proposals and initiatives in my faculty awaiting my decision.
   Signature:
   Name: ViewFacultyPendingApprovals.
   Type: Function.
   Input: @DeanID int.
   Output: Table showing item type, item ID, title, submitter and submission date.

3. Approve, reject, or return an innovation initiative for revision.
   Signature:
   Name: DecideOnInitiative.
   Type: Stored procedure.
   Input: @InitiativeID int, @DeanID int, @Decision varchar(50), @Comments varchar(1000).
   Output: Confirmation message.

4. Escalate an initiative that needs a university-level decision to University Management.

Signature:
Name: EscalateToUniversity.
Type: Stored procedure.
Input: @InitiativeID int, @DeanID int, @Comments varchar(1000).
Output: Confirmation message.

9. View all initiatives in my faculty filtered by status.
   Signature:
   Name: ViewFacultyInitiatives.
   Type: Function.
   Input: @FacultyID int, @Status varchar(50).
   Output: Table of initiatives with course offering, submitter, deadline and status.

10. View all overdue milestones in my faculty.
    Signature:
    Name: ViewOverdueMilestones.
    Type: Function.
    Input: @FacultyID int.
    Output: Table showing initiative, milestone, due date and assigned members.

11. Send overdue alerts to the members assigned to initiatives with overdue milestones.
    Signature:
    Name: SendOverdueAlerts.
    Type: Stored procedure.
    Input: @FacultyID int.
    Output: Confirmation message showing the number of notifications sent.

17. View a dashboard summary of my faculty for a semester.
    Signature:
    Name: GetFacultyDashboard.
    Type: Function.
    Input: @FacultyID int, @SemesterID int.
    Output: Row showing number of active initiatives, pending approvals, overdue milestones and percentage of assessments that met their target.

6 As an University Management member, I should be able to:

1. View all items escalated for a university-level decision.
   Signature:
   Name: ViewEscalatedItems.
   Type: Function.
   Input: Nothing.
   Output: Table showing item type, item ID, title, faculty and escalation date.

2. Record a university-level decision on a proposal or initiative.
   Signature:
   Name: MakeUniversityDecision.
   Type: Stored procedure.
   Input: @ItemType varchar(20), @ItemID int, @ManagerID int, @Decision varchar(50), @Comments varchar(1000).
   Output: Confirmation message.

3. View a university-wide dashboard for a semester.

Signature:
Name: GetUniversityDashboard.
Type: Function.
Input: @SemesterID int.
Output: Table showing each faculty with number of initiatives by status, approval rate and average feedback rating.

4. View initiative statistics by type (AI Tool Integration, Methodology Change, Assessment Redesign).
   Signature:
   Name: GetInitiativeStatsByType.
   Type: Function.
   Input: Nothing.
   Output: Table showing initiative type, status and count.

6. Calculate the average approval turnaround time of a faculty.
   Signature:
   Name: GetApprovalTurnaround.
   Type: Function.
   Input: @FacultyID int.
   Output: Average number of days from submission to final decision int.

9. Rank initiatives by their improvement over baseline for a given metric.
   Signature:
   Name: RankInitiativesByImpact.
   Type: Function.
   Input: @MetricType varchar(50).
   Output: Table of initiatives ordered by percentage improvement over baseline.

7 As an Academic Member, I should be able to:

5. View the initiatives I submitted or am assigned to.
   Signature:
   Name: ViewMyInitiatives.
   Type: Function.
   Input: @MemberID int.
   Output: Table of initiatives with my role, status and deadline.

13. Be notified when a milestone I am responsible for is overdue.
    Signature:
    Name: NotifyOverdueMilestone.
    Type: Stored procedure.
    Input: @MemberID int, @InitiativeID int, @SeqNo int.
    Output: Notification message.